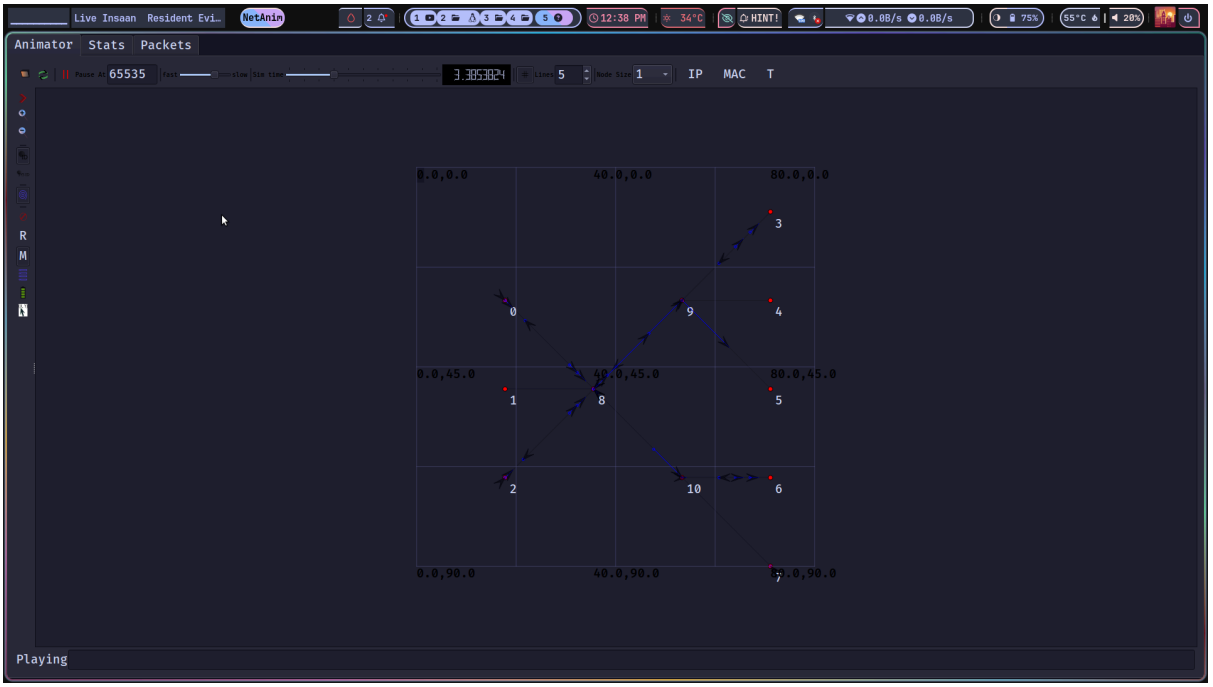
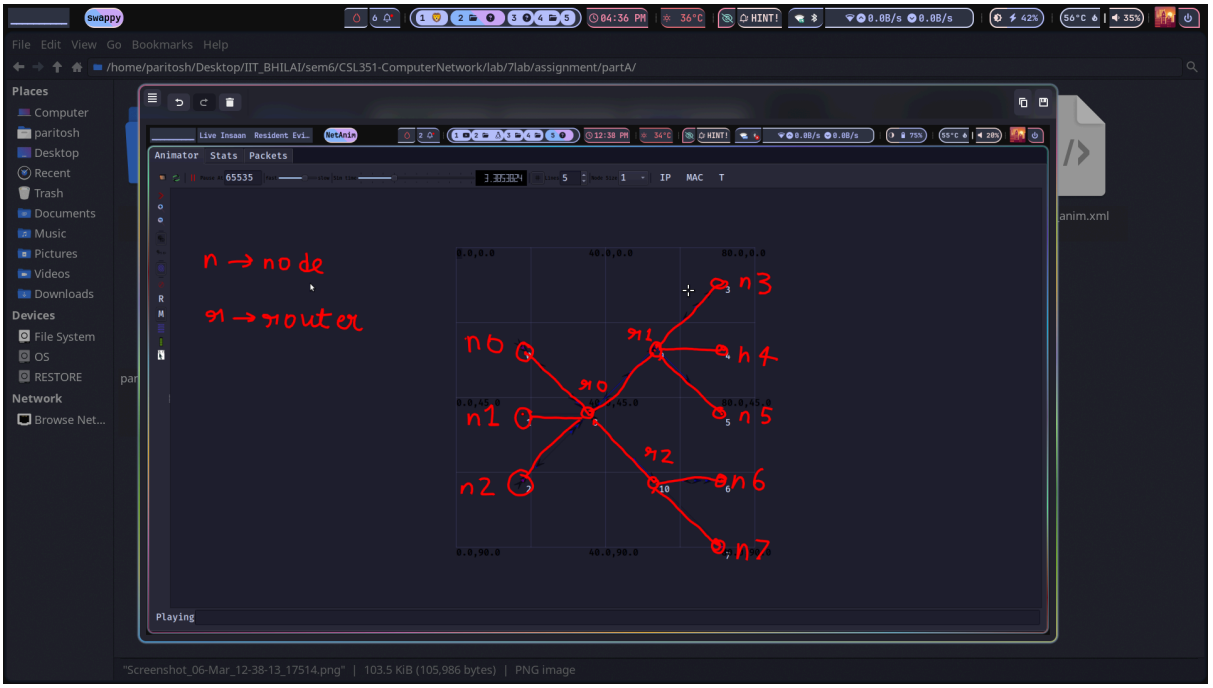
ASSIGNMENT - 06
COMPUTER NETWORKS
ID: 12341550

PART A

pcap:https://drive.google.com/file/d/1C_TFqZXXI_THUWQ-a45p9d_NUQK5eCzc/view?usp=sharing

Command used: p2pStandard.EnablePcapAll("partA_trace");





```
❯ ./ns3 run scratch/partA.cc
[0/2] Re-checking globbed directories...
[2/2] Linking CXX executable /home/parito...s-3.44/build/scratch/ns3.44-partA-default
Flow 1 (10.1.1.1 → 10.1.6.1)
  Tx Packets: 1944
  Rx Packets: 1944
  Lost Packets: 0
  Throughput: 1.04051 Mbps
Flow 2 (10.1.3.1 → 10.1.9.1)
  Tx Packets: 1944
  Rx Packets: 1944
  Lost Packets: 0
  Throughput: 1.04051 Mbps
Flow 3 (10.1.10.1 → 10.1.7.1)
  Tx Packets: 976
  Rx Packets: 976
  Lost Packets: 0
  Throughput: 0.521886 Mbps
Flow 4 (10.1.7.1 → 10.1.10.1)
  Tx Packets: 490
  Rx Packets: 490
  Lost Packets: 0
  Throughput: 0.0242679 Mbps
Flow 5 (10.1.6.1 → 10.1.1.1)
  Tx Packets: 974
  Rx Packets: 974
  Lost Packets: 0
  Throughput: 0.0482217 Mbps
Flow 6 (10.1.9.1 → 10.1.3.1)
  Tx Packets: 974
  Rx Packets: 974
  Lost Packets: 0
  Throughput: 0.0482315 Mbps
Flow 7 (10.1.2.1 → 10.1.8.1)
  Tx Packets: 244
  Rx Packets: 244
  Lost Packets: 0
  Throughput: 0.280073 Mbps
```

Terminal output of **partA.cc**

```
// Flow 1: TCP n7 -> n4
Address tcpSinkAddr1(InetSocketAddress(i_n4_r1.GetAddress(0), port));
PacketSinkHelper tcpSink1("ns3::TcpSocketFactory", tcpSinkAddr1);
ApplicationContainer sinkApp1 = tcpSink1.Install(leafNodes.Get(4));
sinkApp1.Start(Seconds(1.0));
sinkApp1.Stop(Seconds(10.0));

OnOffHelper tcpClient1("ns3::TcpSocketFactory", tcpSinkAddr1);
tcpClient1.SetAttribute("DataRate", DataRateValue(DataRate("1Mbps")));
tcpClient1.SetAttribute("PacketSize", UintegerValue(1024));
ApplicationContainer clientApp1 = tcpClient1.Install(leafNodes.Get(7));
clientApp1.Start(Seconds(2.0));
clientApp1.Stop(Seconds(10.0));
```

TCP flow between **n7 and n4**

```
// Flow 2: UDP n1 -> n5
Address udpSinkAddr2(InetSocketAddress(i_n5_r1.GetAddress(0), port+1));
PacketSinkHelper udpSink2("ns3::UdpSocketFactory", udpSinkAddr2);
ApplicationContainer sinkApp2 = udpSink2.Install(leafNodes.Get(5));
sinkApp2.Start(Seconds(1.0));
sinkApp2.Stop(Seconds(10.0));

OnOffHelper udpClient2("ns3::UdpSocketFactory", udpSinkAddr2);
udpClient2.SetAttribute("DataRate", DataRateValue(DataRate("500Kbps")));
udpClient2.SetAttribute("PacketSize", UintegerValue(1024));
ApplicationContainer clientApp2 = udpClient2.Install(leafNodes.Get(1));
clientApp2.Start(Seconds(2.0));
clientApp2.Stop(Seconds(10.0));
```

UDP flow between **n1 and n5**

```
// Flow 3: TCP n2 -> n6
Address tcpSinkAddr3(InetSocketAddress(i_n6_r2.GetAddress(0), port+2));
PacketSinkHelper tcpSink3("ns3::TcpSocketFactory", tcpSinkAddr3);
ApplicationContainer sinkApp3 = tcpSink3.Install(leafNodes.Get(6));
sinkApp3.Start(Seconds(1.0));
sinkApp3.Stop(Seconds(10.0));

OnOffHelper tcpClient3("ns3::TcpSocketFactory", tcpSinkAddr3);
tcpClient3.SetAttribute("DataRate", DataRateValue(DataRate("2Mbps")));
tcpClient3.SetAttribute("PacketSize", UintegerValue(1024));
ApplicationContainer clientApp3 = tcpClient3.Install(leafNodes.Get(2));
clientApp3.Start(Seconds(2.0));
clientApp3.Stop(Seconds(10.0));
```

TCP flow between **n2 and n6**

```
// Flow 4: TCP n0 -> n3
Address tcpSinkAddr4(InetSocketAddress(i_n3_r1.GetAddress(0), port+3));
PacketSinkHelper tcpSink4("ns3::TcpSocketFactory", tcpSinkAddr4);
ApplicationContainer sinkApp4 = tcpSink4.Install(leafNodes.Get(3));
sinkApp4.Start(Seconds(1.0));
sinkApp4.Stop(Seconds(10.0));

OnOffHelper tcpClient4("ns3::TcpSocketFactory", tcpSinkAddr4);
tcpClient4.SetAttribute("DataRate", DataRateValue(DataRate("2Mbps")));
tcpClient4.SetAttribute("PacketSize", UintegerValue(1024));
ApplicationContainer clientApp4 = tcpClient4.Install(leafNodes.Get(0));
clientApp4.Start(Seconds(2.0));
clientApp4.Stop(Seconds(10.0));
```

TCP flow between **n0 and n3**

```
// Standard Point-to-Point link parameters
PointToPointHelper p2pStandard;
p2pStandard.SetDeviceAttribute("DataRate", StringValue("5Mbps"));
p2pStandard.SetChannelAttribute("Delay", StringValue("2ms"));

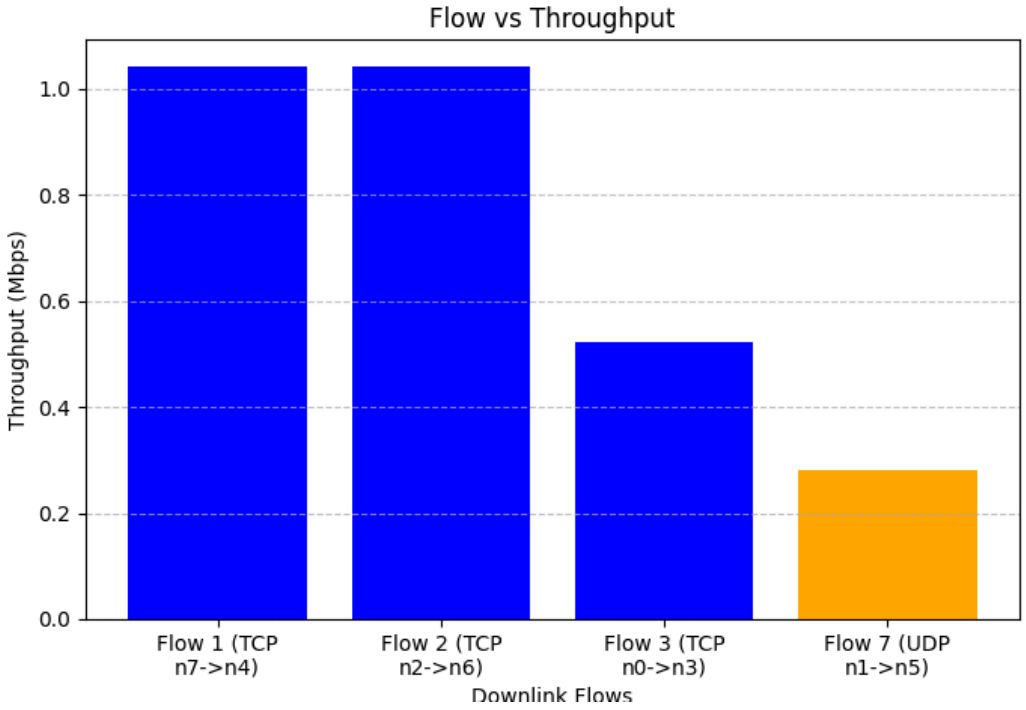
// Special Point-to-Point link parameters for n0->n3 and n2->n6 flows
PointToPointHelper p2pSpecial;
p2pSpecial.SetDeviceAttribute("DataRate", StringValue("10Mbps"));
p2pSpecial.SetChannelAttribute("Delay", StringValue("5ms"));
```

Setting up **delays and data rate**

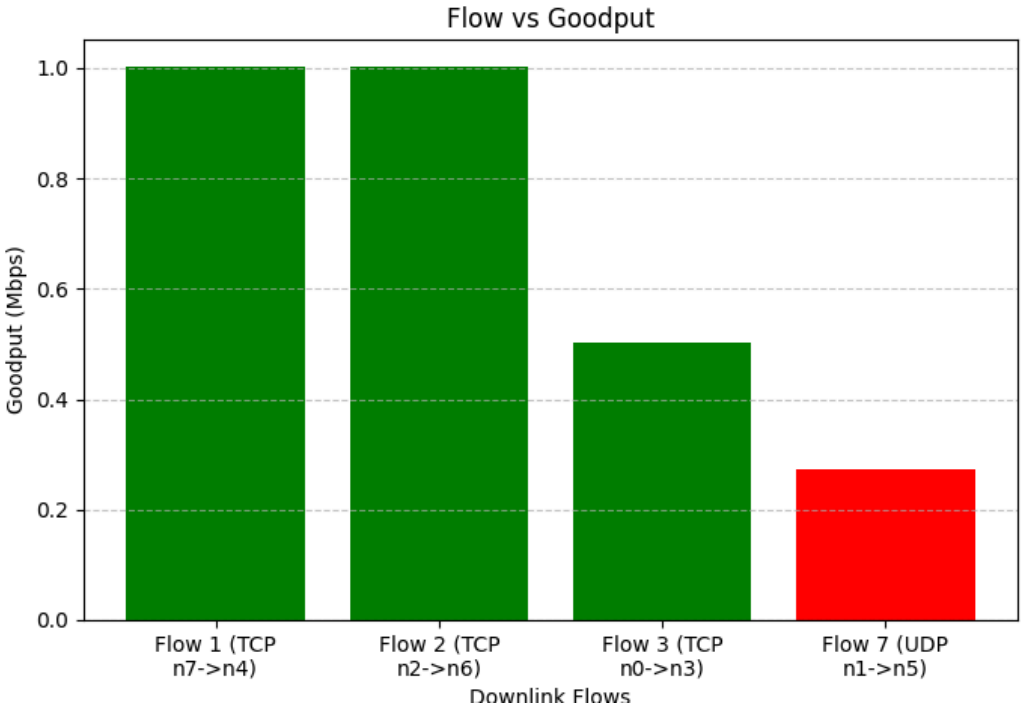
To set the specific data rates and delays for the links, I used the **SetDeviceAttribute("DataRate", ...)** and **SetChannelAttribute("Delay", ...)** functions of the PointToPointHelper class. For IP address assignment, the Ipv4AddressHelper::Assign() function was used to fetch the device interfaces and assign distinct base subnets to each point-to-point link.

TCP and UDP flows were established by installing a **PacketSinkHelpe**r on the receiving nodes and an **OnOffHelpe**r on the sending nodes. The SetAttribute function was utilized to define the specific 'DataRate' and 'PacketSize' for each flow.

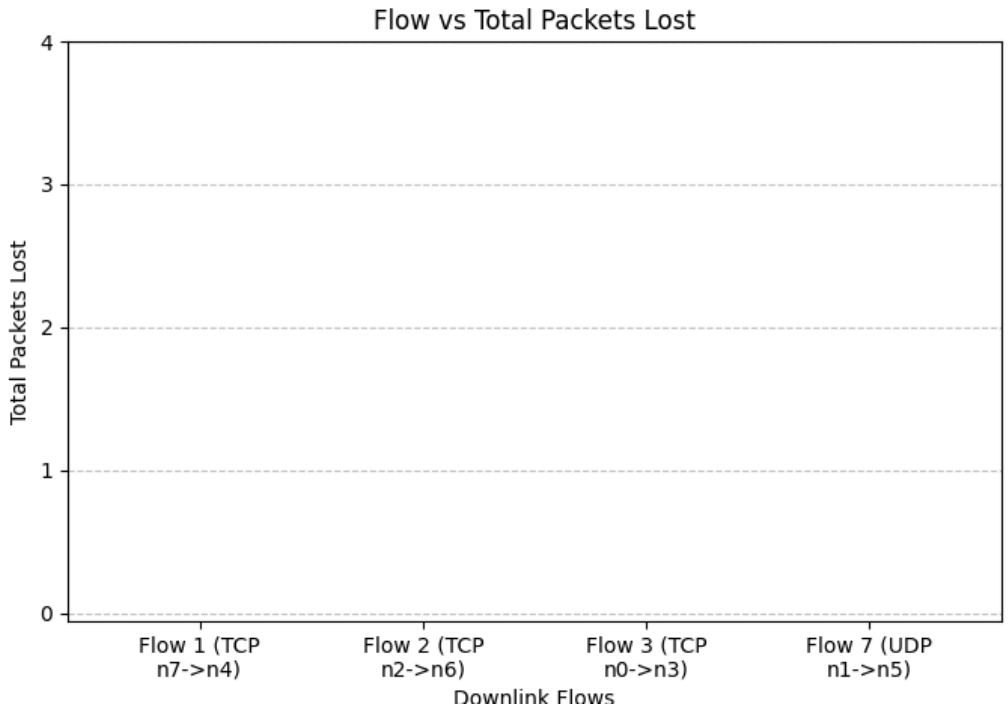
1. **Plot the flow vs throughput graph.**



2. **Plot the flow vs goodput (Not what you get from FlowMon()) graph.**



3. **Plot the flow vs total packets lost graph.**



plot_script.py for generating above graphs

Throughput vs. Goodput: The raw Throughput obtained from FlowMonitor is visibly higher than the calculated Goodput. This is because FlowMonitor records the total bits received at the IP layer (including IP, TCP, and UDP headers). The Goodput was manually calculated by **stripping this 40-byte (TCP) and 28-byte (UDP) overhead** to reflect true application-level data delivery.

Packet Loss: As seen in the 'Flow vs Total Packets Lost' graph , exactly zero packets were dropped. This is the expected behavior because the point-to-point channels in Part A are ideal and no error models have been introduced yet.

4. **Generate a file ‘routing-tables.txt’ wherein you print the routing tables of the 3 routers**.

```
// Populate Routing Tables
Ipv4GlobalRoutingHelper::PopulateRoutingTables();

// Generate routing-tables.txt
Ptr<OutputStreamWrapper> routingStream = Create<OutputStreamWrapper>("routing-tables.txt", std::ios::out);
*routingStream->GetStream() << "Router 0 Routing Table\n";
routers.Get(0)->GetObject<Ipv4>()->GetRoutingProtocol()->PrintRoutingTable(routingStream);
*routingStream->GetStream() << "\nRouter 1 Routing Table\n";
routers.Get(1)->GetObject<Ipv4>()->GetRoutingProtocol()->PrintRoutingTable(routingStream);
*routingStream->GetStream() << "\nRouter 2 Routing Table\n";
routers.Get(2)->GetObject<Ipv4>()->GetRoutingProtocol()->PrintRoutingTable(routingStream);
```

routing-tables.txt

```
Router 0 Routing Table
Node: 8, Time: +0s, Local time: +0s, Ipv4ListRouting table
  Priority: 0 Protocol: ns3::Ipv4StaticRouting
Node: 8, Time: +0s, Local time: +0s, Ipv4StaticRouting table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
127.0.0.0       0.0.0.0         255.0.0.0       U     0      -      -   0
10.1.1.0        0.0.0.0         255.255.255.0   U     0      -      -   1
10.1.2.0        0.0.0.0         255.255.255.0   U     0      -      -   2
10.1.3.0        0.0.0.0         255.255.255.0   U     0      -      -   3
10.1.4.0        0.0.0.0         255.255.255.0   U     0      -      -   4
10.1.5.0        0.0.0.0         255.255.255.0   U     0      -      -   5

  Priority: -10 Protocol: ns3::Ipv4GlobalRouting
Node: 8, Time: +0s, Local time: +0s, Ipv4GlobalRouting table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
10.1.1.1        10.1.1.1        255.255.255.255 UH    -      -      -   1
10.1.2.1        10.1.2.1        255.255.255.255 UH    -      -      -   2
10.1.3.1        10.1.3.1        255.255.255.255 UH    -      -      -   3
10.1.4.2        10.1.4.2        255.255.255.255 UH    -      -      -   4
10.1.6.2        10.1.4.2        255.255.255.255 UH    -      -      -   4
10.1.7.2        10.1.4.2        255.255.255.255 UH    -      -      -   4
10.1.8.2        10.1.4.2        255.255.255.255 UH    -      -      -   4
10.1.5.2        10.1.5.2        255.255.255.255 UH    -      -      -   5
10.1.9.2        10.1.5.2        255.255.255.255 UH    -      -      -   5
10.1.10.2       10.1.5.2        255.255.255.255 UH    -      -      -   5
10.1.6.1        10.1.4.2        255.255.255.255 UH    -      -      -   4
10.1.7.1        10.1.4.2        255.255.255.255 UH    -      -      -   4
10.1.8.1        10.1.4.2        255.255.255.255 UH    -      -      -   4
10.1.9.1        10.1.5.2        255.255.255.255 UH    -      -      -   5
```

routing-tables.txt generated

5. **Generate the NetAnim trace file and append that as well.**

```
// NetAnim
AnimationInterface anim("partA_anim.xml");
```

partA_anim.xml generated

PART B

```
// Flow 1: Echo n7 -> n4
UdpEchoServerHelper server1(port);
ApplicationContainer serverApp1 = server1.Install(leafNodes.Get(4));
serverApp1.Start(Seconds(1.0));
serverApp1.Stop(Seconds(10.0));
UdpEchoClientHelper client1(i_n4_r1.GetAddress(0), port);
client1.SetAttribute("MaxPackets", UintegerValue(150));
client1.SetAttribute("Interval", TimeValue(Seconds(0.05)));
client1.SetAttribute("PacketSize", UintegerValue(1024));
ApplicationContainer clientApp1 = client1.Install(leafNodes.Get(7));
clientApp1.Start(Seconds(2.0));
clientApp1.Stop(Seconds(10.0));
```

EchoClient and EchoServer flow: **n7 to n4**

```
// Flow 2: Echo n1 -> n5
UdpEchoServerHelper server2(port+1);
ApplicationContainer serverApp2 = server2.Install(leafNodes.Get(5));
serverApp2.Start(Seconds(1.0));
serverApp2.Stop(Seconds(10.0));
UdpEchoClientHelper client2(i_n5_r1.GetAddress(0), port+1);
client2.SetAttribute("MaxPackets", UintegerValue(150));
client2.SetAttribute("Interval", TimeValue(Seconds(0.05)));
client2.SetAttribute("PacketSize", UintegerValue(1024));
ApplicationContainer clientApp2 = client2.Install(leafNodes.Get(1));
clientApp2.Start(Seconds(2.0));
clientApp2.Stop(Seconds(10.0));
```

EchoClient and EchoServer flow: **n1 to n5**

```
// Flow 3: Echo n2 -> n6
UdpEchoServerHelper server3(port+2);
ApplicationContainer serverApp3 = server3.Install(leafNodes.Get(6));
serverApp3.Start(Seconds(1.0));
serverApp3.Stop(Seconds(10.0));
UdpEchoClientHelper client3(i_n6_r2.GetAddress(0), port+2);
client3.SetAttribute("MaxPackets", UintegerValue(150));
client3.SetAttribute("Interval", TimeValue(Seconds(0.05)));
client3.SetAttribute("PacketSize", UintegerValue(1024));
ApplicationContainer clientApp3 = client3.Install(leafNodes.Get(2));
clientApp3.Start(Seconds(2.0));
clientApp3.Stop(Seconds(10.0));
```

EchoClient and EchoServer flow: **n2 to n6**

```
// Flow 4: Echo n0 -> n3
UdpEchoServerHelper server4(port+3);
ApplicationContainer serverApp4 = server4.Install(leafNodes.Get(3));
serverApp4.Start(Seconds(1.0));
serverApp4.Stop(Seconds(10.0));
UdpEchoClientHelper client4(i_n3_r1.GetAddress(0), port+3);
client4.SetAttribute("MaxPackets", UintegerValue(150));
client4.SetAttribute("Interval", TimeValue(Seconds(0.05)));
client4.SetAttribute("PacketSize", UintegerValue(1024));
ApplicationContainer clientApp4 = client4.Install(leafNodes.Get(0));
clientApp4.Start(Seconds(2.0));
clientApp4.Stop(Seconds(10.0));
```

EchoClient and EchoServer flow: **n0 to n3**

```
// --- Calculate and Print PDR ---
flowmon->CheckForLostPackets();
std::map<FlowId, FlowMonitor::FlowStats> stats = flowmon->GetFlowStats();

uint64_t t_total = 0;
uint64_t r_total = 0;

for (std::map<FlowId, FlowMonitor::FlowStats>::const_iterator i = stats.begin(); i != stats.end(); ++i) {
    t_total += i->second.txPackets;
    r_total += i->second.rxPackets;
}

double pdr = 0.0;
if (t_total > 0) {
    pdr = (double)r_total / (double)t_total;
}
```

PDR=R_Total/T_Total

```
Ptr<RateErrorModel> em = CreateObject<RateErrorModel>();
em->SetAttribute("ErrorRate", DoubleValue(errorRate));

if (errorUnit == "Packet") {
    em->SetAttribute("ErrorUnit", StringValue("ERROR_UNIT_PACKET"));
} else if (errorUnit == "Bit") {
    em->SetAttribute("ErrorUnit", StringValue("ERROR_UNIT_BIT"));
} else if (errorUnit == "Byte") {
    em->SetAttribute("ErrorUnit", StringValue("ERROR_UNIT_BYTE"));
}

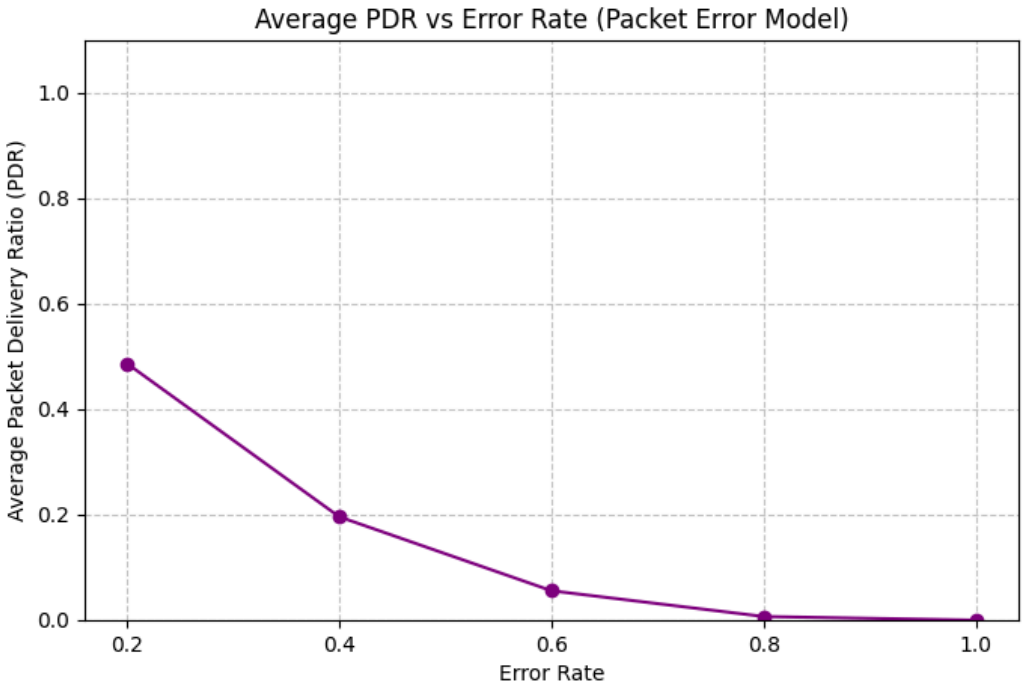
// Safely attach Error Model ONLY to the Point-to-Point devices
NetDeviceContainer allP2PDevices;
allP2PDevices.Add(d_n0_r0);
allP2PDevices.Add(d_n1_r0);
allP2PDevices.Add(d_n2_r0);
allP2PDevices.Add(d_r0_r1);
allP2PDevices.Add(d_r0_r2);
allP2PDevices.Add(d_n3_r1);
allP2PDevices.Add(d_n4_r1);
allP2PDevices.Add(d_n5_r1);
allP2PDevices.Add(d_n6_r2);
allP2PDevices.Add(d_n7_r2);

for (uint32_t i = 0; i < allP2PDevices.GetN(); ++i) {
    allP2PDevices.Get(i)->SetAttribute("ReceiveErrorModel", PointerValue(em));
}
```

The **RateErrorModel** was instantiated and its parameters were tuned using command-line arguments for the ErrorRate and ErrorUnit. It was then attached to the specific **point-to-point devices** using the SetAttribute("ReceiveErrorModel", ...) function to simulate packet corruption.

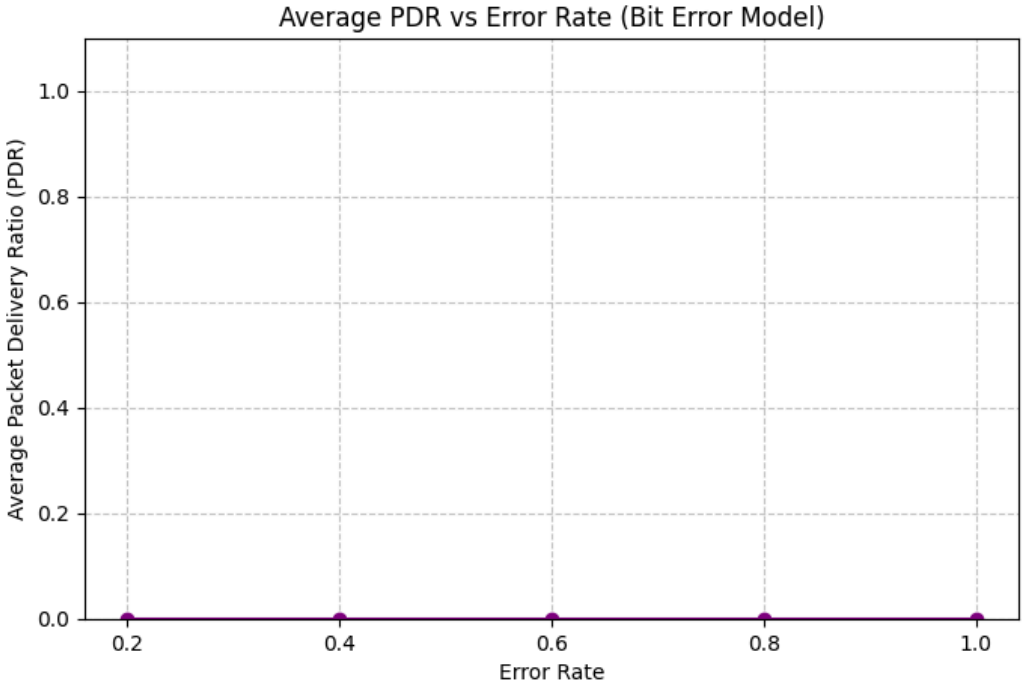
RngRun/Error Rate	ER1=0.2	ER2=0.4	ER3=0.6	ER4=0.8	ER5=1.0
RngRun1=50	0.48434	0.21067	0.0461	0.00332	0
RngRun2=51	0.49213	0.2022	0.05556	0.00498	0
RngRun3=52	0.49164	0.18462	0.06977	0.00498	0
RngRun4=53	0.48318	0.20332	0.0416	0.00498	0
RngRun5=54	0.51827	0.19861	0.05071	0.00662	0
RngRun6=55	0.48706	0.20165	0.0566	0.01153	0
RngRun7=56	0.47133	0.18644	0.06426	0.01153	0
RngRun8=57	0.5	0.21995	0.06552	0.00826	0
RngRun9=58	0.46355	0.16595	0.04928	0.00166	0
RngRun10=59	0.47694	0.18741	0.05827	0.01153	0
AVG. PDR	0.48685	0.19608	0.05577	0.00694	0

table with PDR by transmitting at least 100 packets

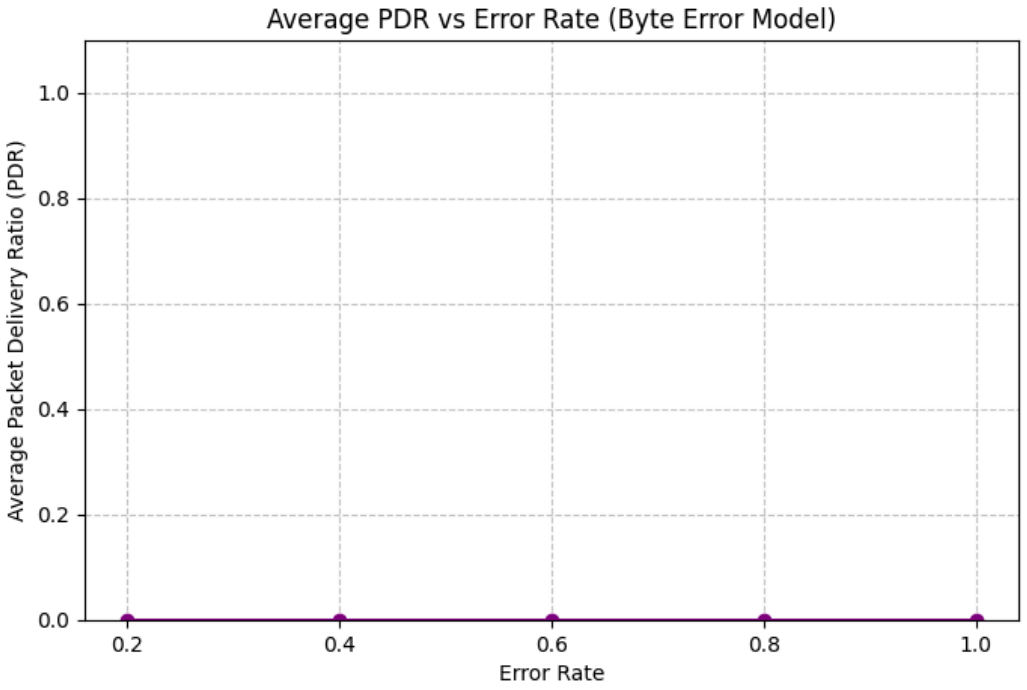


Packet Error Model: The Average PDR vs Error Rate graph for the Packet unit shows a gradual, predictable decline. As the error rate increases from 0.2 to 1.0, the network successfully delivers proportionally fewer packets, dropping from **an average PDR of ~0.48 down to 0**.

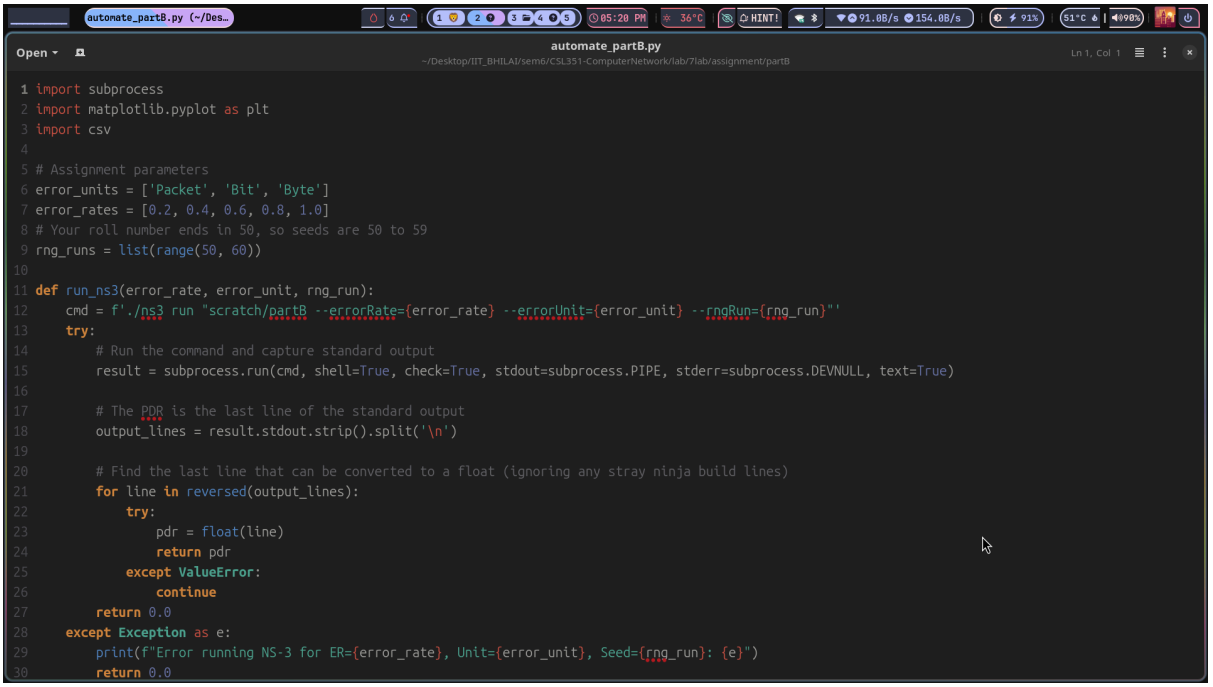
RngRun/Error Rate	ER1=0.2	ER2=0.4	ER3=0.6	ER4=0.8	ER5=1.0
RngRun1=50	0	0	0	0	0
RngRun2=51	0	0	0	0	0
RngRun3=52	0	0	0	0	0
RngRun4=53	0	0	0	0	0
RngRun5=54	0	0	0	0	0
RngRun6=55	0	0	0	0	0
RngRun7=56	0	0	0	0	0
RngRun8=57	0	0	0	0	0
RngRun9=58	0	0	0	0	0
RngRun10=59	0	0	0	0	0
AVG. PDR	0	0	0	0	0



RngRun/Error Rate	ER1=0.2	ER2=0.4	ER3=0.6	ER4=0.8	ER5=1.0
RngRun1=50	0	0	0	0	0
RngRun2=51	0	0	0	0	0
RngRun3=52	0	0	0	0	0
RngRun4=53	0	0	0	0	0
RngRun5=54	0	0	0	0	0
RngRun6=55	0	0	0	0	0
RngRun7=56	0	0	0	0	0
RngRun8=57	0	0	0	0	0
RngRun9=58	0	0	0	0	0
RngRun10=59	0	0	0	0	0
AVG. PDR	0	0	0	0	0



Bit and Byte Error Models: In contrast, the graphs for the Bit and Byte error models show an immediate drop to a PDR of 0 at all tested error rates. This occurs because the RateErrorModel evaluates errors per unit. Since our application payload is 1024 bytes (8192 bits), applying a 20% error rate to individual bits or bytes virtually guarantees that every single packet will contain at least one corrupted unit. A single corrupted bit causes the entire packet to fail the checksum and be dropped, resulting in a 0% delivery ratio.



Script for automation: **automate_partB.py**

```
// --- Command Line Arguments for Automation ---
double errorRate = 0.0;
std::string errorUnit = "Packet";
uint32_t rngRun = 50; // Starting at 50 for my roll number 12341550

CommandLine cmd;
cmd.AddValue("errorRate", "The error rate (0 to 1)", errorRate);
cmd.AddValue("errorUnit", "The error unit (Packet, Bit, Byte)", errorUnit);
cmd.AddValue("rngRun", "The RngRun seed value", rngRun);
cmd.Parse(argc, argv);
```

rngRun value: 50 (my id: 12341**50**)